

Lectures on Monte Carlo Theory

Chapter 4: Simulation Output Analysis: Independent Replications

Based on Leskelä & Vihola

Chapter Outline

- 1 4.1 Estimation of I
- 2 4.2 Estimation of $k(I)$ – Delta Method
- 3 4.3 Estimators of a Quantile
- 4 4.4 Absolute vs Relative Errors
- 5 4.5 Conclusions

The Fundamental Problem

We aim to estimate $I = \mathbb{E}Y$, where Y is a random variable with $\mathbb{E}Y^2 < \infty$ and variance σ_Y^2 . We simulate a sample $Y_1, Y_2, \dots, Y_R$ of i.i.d. random variables.

Two types of errors:

- **Absolute error:** $|\hat{I} - I|$, measures the magnitude.
- **Relative error:** $|\hat{I}/I - 1| = |\hat{I} - I|/I$, evaluates error relative to true value.

Error control conditions (for $\varepsilon, \delta \in (0, 1)$):

$$(E1) \text{ Absolute: } \mathbb{P}(|\hat{I} - I| \geq \varepsilon) \leq \delta$$

$$(E2) \text{ Relative: } \mathbb{P}(|\hat{I}/I - 1| \geq \varepsilon) \leq \delta$$

Chapter Introduction II

Equivalently:

$$(E1'): \mathbb{P}(|\hat{I} - I| \leq \varepsilon) \geq 1 - \delta$$

$$(E2'): \mathbb{P}(|\hat{I} - I| \leq \varepsilon I) \geq 1 - \delta$$

This chapter establishes the *fundamental relationship* between:

- Required precision b
- Number of replications R
- Variability σ_Y^2

4.1.1 One-Dimensional Case: Estimator and CLT I

Sample Mean Estimator

$$\hat{Y}_R = \frac{1}{R} \sum_{j=1}^R Y_j$$

where $Y_1, \dots, Y_R$ are i.i.d. replications of Y with $\mathbb{E}Y = I$.

Properties:

- **Unbiased:** $\mathbb{E}\hat{Y}_R = I$
- **Strongly consistent:** $\hat{Y}_R \rightarrow I$ with probability 1 as $R \rightarrow \infty$
- **Weak consistency:** $\mathbb{P}(|\hat{Y}_R - I| \geq b) \rightarrow 0$ for any $b > 0$

4.1.1 One-Dimensional Case: Estimator and CLT II

Chebyshev-Based Error Bound (Remark 4.1.1)

Using Chebyshev's inequality:

$$b_R = \frac{1}{\alpha^{1/2}} \frac{\sigma_Y}{\sqrt{R}}$$

which means $b_R = O(R^{-1/2})$.

Symbols: b_R = error bound, α = significance level, σ_Y = std. dev. of Y , R = replications.

4.1.1 One-Dimensional Case: Estimator and CLT III

Central Limit Theorem (CLT)

If $Y_1, \dots, Y_R$ are i.i.d. with $\sigma_Y = \sqrt{\text{Var } Y_j} < \infty$, then as $R \rightarrow \infty$:

$$\mathbb{P} \left(\sum_{j=1}^R \frac{Y_j - I}{\sigma_Y \sqrt{R}} \leq x \right) \rightarrow \Phi(x)$$

where $\Phi(x)$ is the standard normal CDF. Equivalently:

$$\sqrt{R}(\hat{Y}_R - I) \xrightarrow{\mathcal{D}} \mathcal{N}(0, \sigma_Y^2)$$

4.1.1 One-Dimensional Case: Estimator and CLT IV

Fundamental Relationship (Eq. 1.6)

$$b = z_{1-\alpha/2} \frac{\sigma_Y}{\sqrt{R}}$$

where $z_{1-\alpha/2} = \Phi^{-1}(1 - \alpha/2)$ is the $(1 - \alpha/2)$ -quantile of $\mathcal{N}(0, 1)$.

Symbols: b = half-width of CI, $z_{1-\alpha/2}$ = normal quantile, σ_Y = std. dev., R = sample size.

4.1.1 One-Dimensional Case: Estimator and CLT V

Required Number of Replications

To achieve prescribed error b at confidence level $1 - \alpha$:

$$R = z_{1-\alpha/2}^2 \frac{\sigma_Y^2}{b^2}$$

Common case ($\alpha = 0.05$, $z_{1-\alpha/2} = z_{0.975} = 1.96$):

$$b = \frac{1.96 \sigma_Y}{\sqrt{R}}, \quad R = \frac{1.96^2 \sigma_Y^2}{b^2}$$

Chebyshev vs CLT comparison (Remark 4.1.5):

$$\frac{b^{\text{CLT}}}{b^{\text{Ch}}} = z_{1-\alpha/2} \alpha^{1/2} = 43.82\% \quad (\alpha = 0.05)$$

Using CLT requires only $\approx 19.2\%$ of the replications needed by Chebyshev's bound.

4.1.1 Sample Variance and Confidence Interval I

In practice σ_Y^2 is unknown; we replace it with the **sample variance**:

Unbiased Sample Variance

$$\hat{S}_R^2 = \frac{1}{R-1} \sum_{j=1}^R (Y_j - \hat{Y}_R)^2$$

$\mathbb{E}\hat{S}_R^2 = \sigma_Y^2$ (unbiased), strongly consistent.

$1 - \alpha$ Confidence Interval (Eq. 1.9)

$$\left(\hat{Y}_R - z_{1-\alpha/2} \frac{\hat{S}_R}{\sqrt{R}}, \hat{Y}_R + z_{1-\alpha/2} \frac{\hat{S}_R}{\sqrt{R}} \right)$$

Interpretation: Contains the true I with approximate probability $1 - \alpha$.

4.1.1 Sample Variance and Confidence Interval II

Recursive update formulas (online computation):

$$\hat{Y}_{R+1} = \hat{Y}_R + \frac{Y_{R+1} - \hat{Y}_R}{R+1}$$

$$\hat{\sigma}_{R+1}^2 = \hat{\sigma}_R^2 + (\hat{Y}_R)^2 - (\hat{Y}_{R+1})^2 + \frac{(Y_{R+1})^2 - \hat{\sigma}_R^2 - (\hat{Y}_R)^2}{R+1}$$

4.1.1 Examples: Estimating π I

Example 4.1.6 – Indicator (Hit-or-Miss) Estimator

Take $Y = 4 \cdot \mathbf{1}(U_1^2 + U_2^2 \leq 1)$ where $U_1, U_2 \sim \mathcal{U}[0, 1]$ i.i.d.

$$\hat{Y}_R = \frac{1}{R} \sum_{j=1}^R Y_j, \quad Y_j = 4 \cdot \mathbf{1}(U_{2j-1}^2 + U_{2j}^2 \leq 1)$$

$\mathbb{E}Y = \pi$, $\text{Var } Y = 16(\pi/4)(1 - \pi/4) \approx 2.6968$.

For 1st digit accuracy ($b = 0.5$): $R \approx 1.96^2 \cdot 2.6968 / 0.25 \approx 42$.

For 2nd digit ($b = 0.05$): $R \approx 4144$.

4.1.1 Examples: Estimating π II

Example 4.1.7 – Smooth Estimator

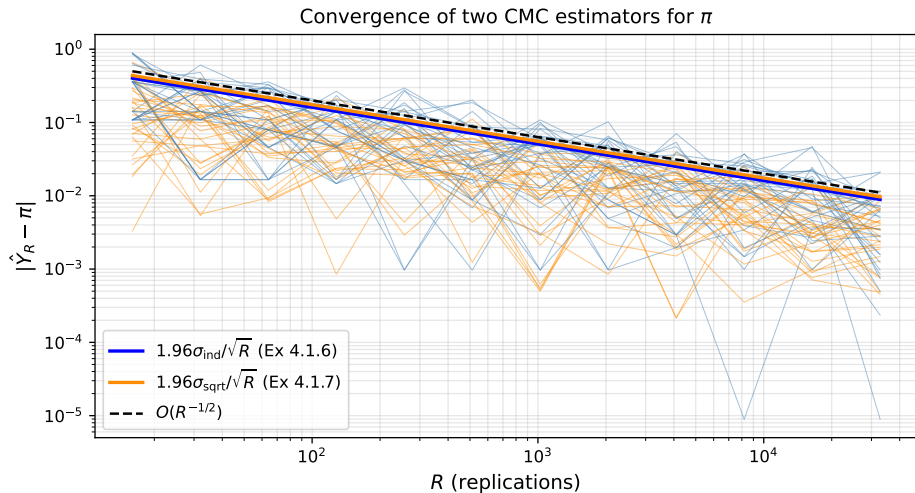
Take $Y = 4\sqrt{1 - U^2}$ where $U \sim \mathcal{U}[0, 1]$, so $\mathbb{E}Y = \pi$.

$$\text{Var } Y = \frac{32}{3} - \pi^2 \approx 0.797$$

For 1st digit ($b = 0.5$): $R \approx 12$. For 2nd digit ($b = 0.05$): $R \approx 1224$.

Variance $\approx 3.38\times$ smaller than Example 4.1.6!

4.1.1 Examples: Estimating π III



4.1.1 Crude Monte Carlo and Hit-or-Miss

Crude Monte Carlo (CMC)

For $I = \int_B k(x) dx$ with $\text{Vol}(B) = 1$:

$$\hat{Y}_R^{\text{CMC}} = \frac{1}{R} \sum_{j=1}^R k(V_j)$$

where $V_j \sim \mathcal{U}(B)$ i.i.d.

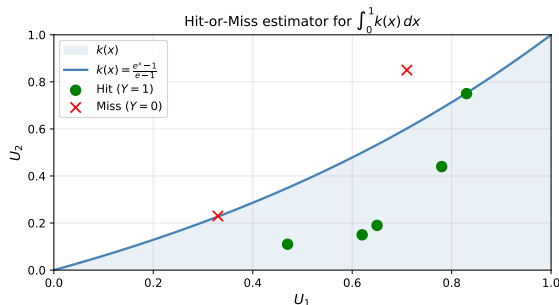
Hit-or-Miss (HoM)

For $0 \leq k(x) \leq 1$, define:

$$Y' = \mathbf{1}(k(U_1) > U_2)$$

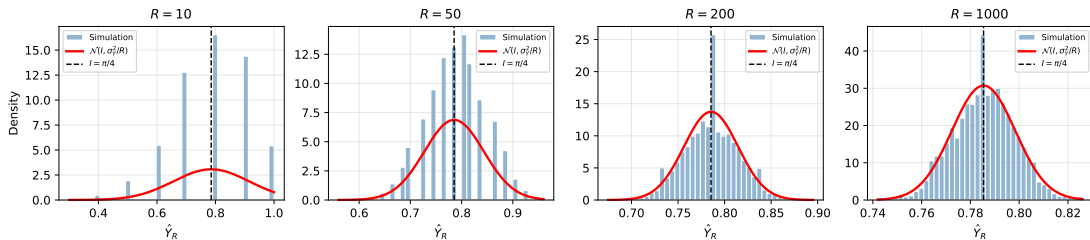
Then $\mathbb{E}Y' = \int_0^1 k(u) du = I$.

$$\hat{Y}_{\text{HoM}} = \frac{1}{R} \sum_{j=1}^R Y'_j$$



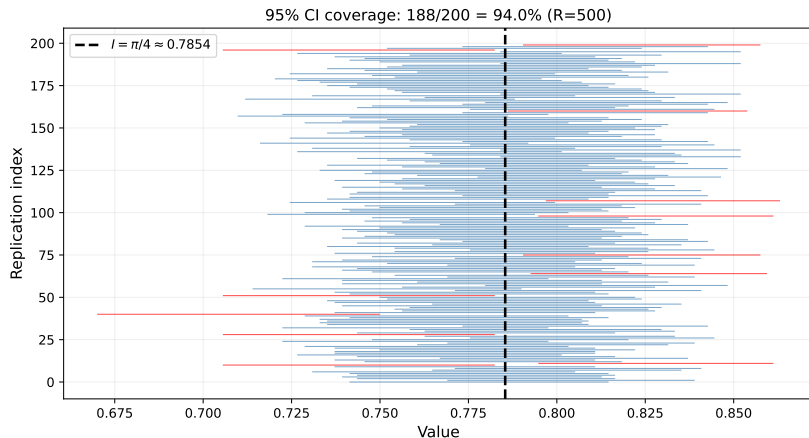
4.1.1 CLT: Distribution of $\hat{Y}_R$

CLT: Distribution of $\hat{Y}_R$ for estimating $I = \pi/4$



- As R increases, $\hat{Y}_R$ concentrates around $I = \pi/4$
- Distribution approaches $\mathcal{N}(I, \sigma_Y^2/R)$ (red curve)
- CI coverage confirmed empirically

4.1.1 Confidence Interval Coverage



- 200 independent 95% CIs for $I = \pi/4$ with $R = 500$
- Blue: CI contains I (correct); Red: CI misses I
- Coverage close to nominal 95%

4.1.1 Python: Computing a Confidence Interval

```
import numpy as np
from scipy import stats

def estimate_pi_ci(R, alpha=0.05, seed=42):
    """Estimate pi using hit-or-miss MC and compute 95% CI."""
    rng = np.random.default_rng(seed)
    U = rng.random((R, 2))
    #  $Y_j = 4 * 1(U_1^2 + U_2^2 \leq 1)$ 
    Y = 4.0 * (U[:, 0]**2 + U[:, 1]**2 <= 1).astype(float)

    Y_bar = Y.mean()           # estimator
    S2 = Y.var(ddof=1)         # unbiased sample variance
    S = np.sqrt(S2)

    z = stats.norm.ppf(1 - alpha / 2)  #  $z_{1-\alpha/2}$ 
    margin = z * S / np.sqrt(R)

    ci_lo = Y_bar - margin
    ci_hi = Y_bar + margin
    return Y_bar, ci_lo, ci_hi

# Example: R = 10000, alpha = 0.05
Y_bar, lo, hi = estimate_pi_ci(R=10_000)
print(f"Estimate: {Y_bar:.4f}")
print(f"95% CI: ({lo:.4f}, {hi:.4f})")
print(f"True pi: {np.pi:.4f}")
# Output: Estimate: 3.1464, 95% CI: (3.1142, 3.1786)
```

4.1.2 Multidimensional Case I

We aim to estimate $\mathbf{I} = (I^{(1)}, \dots, I^{(d)})^T = \mathbb{E}\mathbf{Y}$ where $\mathbf{Y} = (Y^{(1)}, \dots, Y^{(d)})^T \in \mathbb{R}^d$.

Vector Sample Mean Estimator

$$\hat{\mathbf{Y}}_R = \frac{1}{R} \sum_{j=1}^R \mathbf{Y}_j, \quad \mathbf{Y}_j = \begin{pmatrix} Y_j^{(1)} \\ \vdots \\ Y_j^{(d)} \end{pmatrix}$$

Multivariate CLT

Let $\Sigma_{\mathbf{Y}} = \mathbb{E}[(\mathbf{Y} - \mathbf{I})(\mathbf{Y} - \mathbf{I})^T]$ be the covariance matrix of $\mathbf{Y}$. Then:

$$\sqrt{R}(\hat{\mathbf{Y}}_R - \mathbf{I}) \xrightarrow{\mathcal{D}} \mathcal{N}(\mathbf{0}, \Sigma_{\mathbf{Y}})$$

so $\hat{\mathbf{Y}}_R \approx \mathcal{N}(\mathbf{I}, \frac{1}{R}\Sigma_{\mathbf{Y}})$.

4.1.2.1 Facts about the Multivariate Normal Distribution I

Definition

$\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ with $\boldsymbol{\Sigma}$ nonsingular $d \times d$ covariance matrix.

Spectral Decomposition

$\boldsymbol{\Sigma} = \mathbf{F}\boldsymbol{\Lambda}\mathbf{F}^T = \sum_{i=1}^d \lambda_i \mathbf{f}_i \mathbf{f}_i^T$ where:

- $\lambda_1, \dots, \lambda_d > 0$: eigenvalues of $\boldsymbol{\Sigma}$
- $\mathbf{f}_1, \dots, \mathbf{f}_d$: orthonormal eigenvectors
- $\boldsymbol{\Sigma}^{1/2} = \mathbf{F}\boldsymbol{\Lambda}^{1/2}\mathbf{F}^T$ (square root)

4.1.2.1 Facts about the Multivariate Normal Distribution II

Proposition 4.1.13 – Key Properties

For $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ with $\boldsymbol{\Sigma} = \mathbf{A}\mathbf{A}^T$:

- (a) $\mathbf{Z} = \mathbf{A}^{-1}(\mathbf{X} - \boldsymbol{\mu}) \sim \mathcal{N}(\mathbf{0}, \mathbf{Id}_d)$
- (b) $(\mathbf{X} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{X} - \boldsymbol{\mu}) \sim \chi_d^2$ (chi-square with d d.f.)

4.1.2.1 Facts about the Multivariate Normal Distribution III

Consequence: For $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$:

$$\mathbb{P}((\mathbf{X} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{X} - \boldsymbol{\mu}) \leq q_{1-\alpha}(\chi_d^2)) \approx 1 - \alpha$$

where $q_{1-\alpha}(\chi_d^2)$ is the $(1 - \alpha)$ -quantile of χ_d^2 .

Example values ($\alpha = 0.05$):

d	$q_{0.95}(\chi_d^2)$	Ellipsoid “radius”
1	3.841	1.96
2	5.991	—
5	11.07	—

4.1.2.2 Shapes of Confidence Regions I

For $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, three standard confidence region shapes:

Ellipsoid (Tightest)

$$\mathcal{C}^{\text{el}} = \{\mathbf{x} : \mathbf{x}^T \boldsymbol{\Sigma}^{-1} \mathbf{x} \leq q_{1-\alpha}(\chi_d^2)\}$$

Centered at $\mathbf{0}$ with principal axes $\sqrt{q_{1-\alpha}(\chi_d^2) \lambda_i} \mathbf{f}_i$.

Parallelogram (via Hypercube)

Define hypercube $\mathcal{H} = (-z, z)^d$ where $z = \Phi^{-1}\left(\frac{\sqrt[d]{1-\alpha}+1}{2}\right)$. The confidence parallelogram is:

$$\mathcal{C}^{\text{par}} = \boldsymbol{\Sigma}^{1/2} \mathcal{H} + \boldsymbol{\mu}$$

with vertices $\boldsymbol{\Sigma}^{1/2} \mathbf{v}_i + \boldsymbol{\mu}$ for $\mathbf{v}_i = (\pm z, \dots, \pm z)$.

4.1.2.2 Shapes of Confidence Regions II

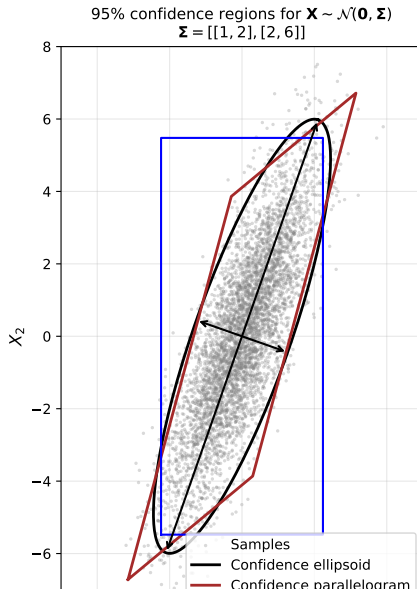
Rectangle (Conservative, via Sidak)

By independence of coordinates of $\Sigma^{-1/2}(\mathbf{X} - \boldsymbol{\mu})$:

$$\mathcal{C}^{\text{rec}} = [-c_1, c_1] \times \cdots \times [-c_d, c_d], \quad c_i = \sigma_i \Phi^{-1} \left(\frac{1 + \sqrt[d]{1 - \alpha}}{2} \right)$$

This uses the *Sidak inequality*: satisfies $\mathbb{P}(\mathbf{X} \in \mathcal{C}^{\text{rec}}) \geq 1 - \alpha$.

4.1.2.2 Shapes of Confidence Regions III



4.1.2.3 Confidence Regions for $\hat{\mathbf{Y}}_R$ of $\mathbf{I}$

We now construct confidence regions for the *estimator* $\hat{\mathbf{Y}}_R$ (centered at $\hat{\mathbf{Y}}_R$, covering $\mathbf{I}$).

Estimated Confidence Ellipsoid

Procedure:

- 1 Compute $\hat{\mathbf{Y}}_R = \frac{1}{R} \sum_{k=1}^R \mathbf{Y}_k$
- 2 Compute sample covariance $\hat{\mathbf{S}}_R^2 = \frac{1}{R-1} \sum_{k=1}^R (\mathbf{Y}_k - \hat{\mathbf{Y}}_R)(\mathbf{Y}_k - \hat{\mathbf{Y}}_R)^T$
- 3 Compute eigendecomposition of $\hat{\mathbf{S}}_R^2$: eigenvalues λ_i , eigenvectors $\mathbf{f}_i$
- 4 Principal axes of ellipsoid: $\sqrt{\frac{q_{1-\alpha}(\chi_d^2) \lambda_i}{R}} \mathbf{f}_i$
- 5 Ellipsoid: $\hat{\mathbf{Y}}_R + \mathcal{C}$

Note (Remark 4.1.15): Replacing $\Sigma_{\mathbf{Y}}$ by $\hat{\mathbf{S}}_R^2$ means the statistic $R(\hat{\mathbf{Y}}_R - \mathbf{I})^T (\hat{\mathbf{S}}_R^2)^{-1} (\hat{\mathbf{Y}}_R - \mathbf{I})$ follows Hotelling's T^2 -distribution. For large R , this is approximately χ_d^2 .

4.1.2.3 Confidence Regions for $\hat{\mathbf{Y}}_R$ of $\mathbf{I}$ II

Example 4.1.16 – Heights/Weights dataset ($R = 1000$, $d = 2$: height $I^{(1)}$, weight $I^{(2)}$ of 18-year-olds):

- $\hat{Y}_R^{(1)} = 127.48$, $\hat{S}_R^{(1)} = 11.80$
- $\hat{Y}_R^{(2)} = 68.05$, $\hat{S}_R^{(2)} = 1.94$
- 95% CI for height: $\mathbb{P}(126.75 \leq I^{(1)} \leq 128.21) = 95\%$
- 95% CI for weight: $\mathbb{P}(67.93 \leq I^{(2)} \leq 68.17) = 95\%$

Sample Covariance Matrix

$$\hat{\mathbf{S}}_R^2 = \begin{pmatrix} 139.29 & 11.40 \\ 11.40 & 3.75 \end{pmatrix}, \quad \lambda_1 = 140.24, \lambda_2 = 2.79$$

4.1.2 Python: Multivariate Confidence Ellipsoid

```
import numpy as np
from scipy.stats import chi2

def conf_ellipsoid(Y_samples, alpha=0.05):
    """Compute confidence ellipsoid for mean of d-dim samples."""
    R, d = Y_samples.shape
    Y_bar = Y_samples.mean(axis=0)           # sample mean (d-vector)

    # Sample covariance matrix
    diff = Y_samples - Y_bar
    S2 = (diff.T @ diff) / (R - 1)

    # Eigendecomposition
    eigvals, eigvecs = np.linalg.eigh(S2)
    q = chi2.ppf(1 - alpha, df=d)           # chi-square quantile

    # Axis lengths of confidence ellipsoid
    axis_lengths = np.sqrt(q * eigvals / R)
    return Y_bar, axis_lengths, eigvecs, S2

# Simulate 2D example: I = (2.0, 3.0), Sigma = [[1,0.5],[0.5,2]]
rng = np.random.default_rng(42)
Sigma = np.array([[1., 0.5], [0.5, 2.]])
L = np.linalg.cholesky(Sigma)
R = 500
Y = (L @ rng.standard_normal((2, R))).T + np.array([2.0, 3.0])
```

4.2 Estimation of $k(I)$ – Overview I

Problem Setup

We aim to estimate $z = k(I)$ (or $z = k(\mathbf{I})$), where:

- $k : \mathbb{R}^d \rightarrow \mathbb{R}$ is a prescribed continuous function
- $I = \mathbb{E}Y$ (or $\mathbf{I} = \mathbb{E}\mathbf{Y}$) is computable via simulation
- $\hat{X}_R$ is a strongly consistent estimator of I

The natural estimator is:

$$\hat{Z}_R = k(\hat{X}_R)$$

Key quantity – Asymptotic Variance:

$$\varsigma^2 = \lim_{R \rightarrow \infty} R \text{Var} \hat{Z}_R \quad (\text{also called the TAVC})$$

4.2 Estimation of $k(I)$ – Overview II

Example: $k(x) = x$ (identity)

$\hat{Z}_R = \hat{Y}_R$, $\varsigma^2 = \text{Var } Y = \sigma_Y^2$. This reduces to Section 4.1.

Important: $k(\hat{Y}_R)$ is generally *biased* even when $\hat{Y}_R$ is unbiased.

4.2.1 The Case $\hat{X}_R = \hat{Y}_R$ – Delta Method I

Proposition 4.2.1 – Delta Method (1D)

Let $Y_1, Y_2, \dots$ be i.i.d., $\hat{Y}_R = \frac{1}{R} \sum Y_j$, and suppose k is continuously differentiable in a neighborhood of $I = \mathbb{E}Y$. Then:

$$\sqrt{R}(k(\hat{Y}_R) - k(I)) \xrightarrow{\mathcal{D}} \mathcal{N}(0, \varsigma^2)$$

where:

$$\varsigma^2 = k'(I)^2 \sigma_Y^2 = \lim_{R \rightarrow \infty} R \text{Var} \hat{Z}_R$$

Symbols: $k'(I)$ = derivative of k at I , $\sigma_Y^2 = \text{Var} Y$.

Proof sketch (Delta Method): Taylor expansion:

$$k(x) - k(I) = k'(I)(x - I) + o(x - I)$$

So $\sqrt{R}(k(\hat{Y}_R) - k(I)) \approx k'(I)\sqrt{R}(\hat{Y}_R - I) \xrightarrow{\mathcal{D}} \mathcal{N}(0, k'(I)^2 \sigma_Y^2)$.

4.2.1 The Case $\hat{X}_R = \hat{Y}_R$ – Delta Method II

Practical Confidence Interval for $k(I)$ (Eq. 2.3)

$$\left(k(\hat{Y}_R) - z_{1-\alpha/2} \frac{\hat{S}_R k'(\hat{Y}_R)}{\sqrt{R}}, k(\hat{Y}_R) + z_{1-\alpha/2} \frac{\hat{S}_R k'(\hat{Y}_R)}{\sqrt{R}} \right)$$

where $\hat{S}_R^2$ is the sample variance and $k'(\hat{Y}_R)$ approximates $k'(I)$.

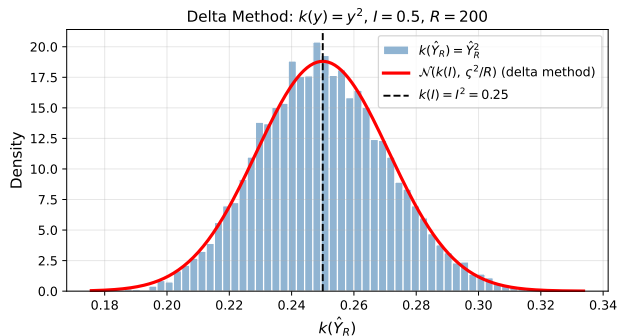
Example 4.2.2 – Insurance Net Premium

$\Pi = r \frac{\mathbb{E}e^{-rT}}{1 - \mathbb{E}e^{-rT}} = k(I)$ where $I = \mathbb{E}e^{-rT}$ and $k(x) = \frac{rx}{1-x}$, $k'(x) = \frac{r}{(1-x)^2}$.

Confidence interval:

$$\left(k(\hat{Y}_R) - \frac{z_{1-\alpha/2} r \hat{S}_R}{(1 - \hat{Y}_R)^2 \sqrt{R}}, k(\hat{Y}_R) + \frac{z_{1-\alpha/2} r \hat{S}_R}{(1 - \hat{Y}_R)^2 \sqrt{R}} \right)$$

4.2.1 The Case $\hat{X}_R = \hat{Y}_R$ – Delta Method III



4.2.1 Multidimensional Delta Method I

Proposition 4.2.3 – 2D Case

Let $\mathbf{Y}_1, \mathbf{Y}_2, \dots$ be i.i.d. replications of $\mathbf{Y} = (Y^{(1)}, Y^{(2)})^T$, and $k(y_1, y_2)$ continuously differentiable at $\mathbf{I} = (I^{(1)}, I^{(2)})$. For $\hat{Z}_R = k(\hat{\mathbf{Y}}_R)$:

$$\sqrt{R}(k(\hat{\mathbf{Y}}_R) - k(\mathbf{I})) \xrightarrow{\mathcal{D}} \mathcal{N}(0, \varsigma^2)$$

where:

$$\varsigma^2 = (k'_{y_1})^2 \Sigma_{\mathbf{Y}}(1, 1) + (k'_{y_2})^2 \Sigma_{\mathbf{Y}}(2, 2) + 2k'_{y_1} k'_{y_2} \Sigma_{\mathbf{Y}}(1, 2)$$

(all partial derivatives evaluated at $\mathbf{I}$).

4.2.1 Multidimensional Delta Method II

General Case – Proposition 4.2.8

For $k : \mathbb{R}^d \rightarrow \mathbb{R}$ continuously differentiable at $\mathbf{I}$:

$$\sqrt{R}(k(\hat{\mathbf{Y}}_R) - k(\mathbf{I})) \xrightarrow{\mathcal{D}} \mathcal{N}(0, \varsigma^2)$$

$$\varsigma^2 = \nabla k(\mathbf{I}) \boldsymbol{\Sigma}_{\mathbf{Y}} (\nabla k(\mathbf{I}))^T = \sum_{i,j=1}^d k^{(i)}(\mathbf{I}) k^{(j)}(\mathbf{I}) \boldsymbol{\Sigma}_{\mathbf{Y}}(i, j)$$

4.2.1 Multidimensional Delta Method III

Practical CI for $z = k(\mathbf{I})$ at level $1 - \alpha$:

$$k(\hat{\mathbf{Y}}_R) \pm 1.96 \frac{\hat{\varsigma}}{\sqrt{R}}$$

where estimated asymptotic variance:

$$\hat{\varsigma}^2 = \sum_{i,j=1}^d k^{(i)}(\hat{\mathbf{Y}}_R) k^{(j)}(\hat{\mathbf{Y}}_R) \hat{\mathbf{S}}_R^2(i, j)$$

4.2.1 Multidimensional Delta Method IV

Example 4.2.5 – Ratio of Expectations

Estimate $z = I^{(1)}/I^{(2)}$ using $k(y_1, y_2) = y_1/y_2$:

$$\hat{Z}_R = \frac{\hat{Y}_R^{(1)}}{\hat{Y}_R^{(2)}}, \quad \varsigma^2 = \frac{\mathbb{E}(Y^{(1)} - zY^{(2)})^2}{(I^{(2)})^2}$$

Estimated variance:

$$\widehat{\text{Var}}(\hat{Z}_R) = \frac{1}{R^2(\hat{Y}_R^{(2)})^2} \sum_{i=1}^R (Y_i^{(1)} - \hat{Z}_R Y_i^{(2)})^2$$

4.2.2 General Consistent Estimator $\hat{X}_R$ I

Consider a more general strongly consistent estimator $\hat{X}_R$ of I satisfying:

- ① $\sqrt{R}(\hat{X}_R - \mathbb{E}\hat{X}_R) \xrightarrow{\mathcal{D}} \mathcal{N}(0, \sigma^2)$, where $0 < \sigma^2 < \infty$
- ② $\sqrt{R}(\mathbb{E}\hat{X}_R - I) \rightarrow 0$ (asymptotic unbiasedness)

Result: Under (A1) and (A2):

$$\sqrt{R}(\hat{X}_R - I) \xrightarrow{\mathcal{D}} \mathcal{N}(0, \sigma^2)$$

and for $\hat{Z}_R = k(\hat{X}_R)$:

$$\sqrt{R}(k(\hat{X}_R) - k(I)) \xrightarrow{\mathcal{D}} \mathcal{N}(0, \varsigma^2), \quad \varsigma^2 = k'(I)^2 \sigma^2$$

4.2.2 General Consistent Estimator $\hat{X}_R$ II

Asymptotic Bias of $\hat{Z}_R$ (Eq. 2.17)

Under stronger assumption (A2'): $R(\mathbb{E}\hat{X}_R - I) \rightarrow \beta^X$, the R^{-1} -order asymptotic bias of $\hat{Z}_R$ is:

$$\beta^Z = \beta^X k'(I) + \frac{\sigma^2 k''(I)}{2}$$

Bias-corrected estimator: $k(\hat{X}_R) - \beta^Z/R$.

4.2.3 Mean Square Error, Bias, and Variance I

Bias-Variance Decomposition (MSE)

For estimator $\hat{X}_R$ of I :

$$\text{MSE}(\hat{X}_R) = \mathbb{E}(\hat{X}_R - I)^2 = \underbrace{\mathbb{E}(\hat{X}_R - \mathbb{E}\hat{X}_R)^2}_{\text{Variance}} + \underbrace{(\mathbb{E}\hat{X}_R - I)^2}_{\text{Bias}^2}$$

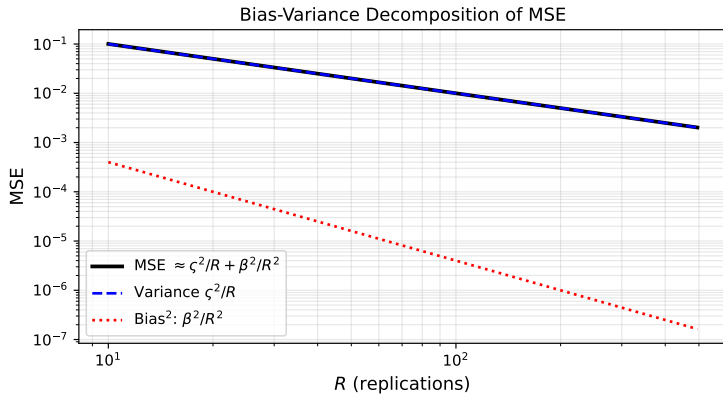
This is the *bias-variance decomposition* (Eq. 2.19).

For large R :

$$\text{MSE}(\hat{X}_R) \approx \frac{\varsigma^2}{R} + \frac{\beta^2}{R^2}$$

- **Variance term** ς^2/R dominates for large R (order R^{-1})
- **Bias² term** β^2/R^2 is negligible for large R (order R^{-2})

4.2.3 Mean Square Error, Bias, and Variance II



4.2.3 Mean Square Error, Bias, and Variance III

Rule of Thumb (Remark 4.2.4)

Bias corrections are often implemented believing they improve accuracy in small samples. However, for large R (typical in Monte Carlo), the standard deviation $\varsigma/\sqrt{R}$ dominates the bias β/R , making correction negligible.

Principle: The standard deviation should dominate the bias for confidence intervals to remain meaningful.

Example 4.2.9 – $k(y) = y^2$

For $\hat{Z}_R = \hat{Y}_R^2$:

$$\mathbb{E}(\hat{Y}_R)^2 = (\mathbb{E}Y)^2 + R^{-1}\sigma_Y^2 = k(I) + R^{-1}\sigma_Y^2$$

The asymptotic bias of order R^{-1} is $\beta = \sigma_Y^2/2$ (from $k''(y) = 2$, since $\beta = k''(I)\sigma_Y^2/2$).

4.2 Python: Delta Method Confidence Interval

```
import numpy as np
from scipy import stats

def delta_method_ci(Y_samples, k_func, k_prime, alpha=0.05):
    """
    Compute CI for  $k(I)$  using delta method.

    Parameters
    -----
    Y_samples : 1D array of i.i.d. realizations of  $Y$  ( $EY = I$ )
    k_func     : callable, the transformation  $k$ 
    k_prime    : callable, derivative  $k'$ 
    """
    R = len(Y_samples)
    Y_bar = Y_samples.mean()           # estimator of  $I$ 
    S = Y_samples.std(ddof=1)          # sample std dev

    z_est = k_func(Y_bar)              # point estimate of  $k(I)$ 
    # Asymptotic std dev:  $|k'(Y_{\text{bar}})| * S / \sqrt{R}$ 
    asymp_std = abs(k_prime(Y_bar)) * S / np.sqrt(R)

    z_q = stats.norm.ppf(1 - alpha / 2)
    return z_est, z_est - z_q * asymp_std, z_est + z_q * asymp_std

# Example:  $k(y) = y^2$ ,  $Y \sim N(0.5, 0.09)$ ,  $I=0.5$ ,  $k(I)=0.25$ 
rng = np.random.default_rng(42)
Y = rng.normal(0.5, 0.3, 1000)
```

4.3 Estimators of a Quantile I

Setup

Let $X \sim F$ with density f . The p -th quantile (for $p \in (0, 1)$) is:

$$q_p = F^{\leftarrow}(p) = \inf\{x : F(x) \geq p\}$$

In financial math: $q_p = \text{VaR}_X(p)$ (Value-at-Risk).

Sample Quantile via Order Statistics

Given i.i.d. sample $X_1, \dots, X_R$ with order statistics $X_{(1)} \leq \dots \leq X_{(R)}$:

$$\hat{F}_R^{\leftarrow}(p) = \begin{cases} X_{(\lfloor pR \rfloor)} & \text{if } pR \text{ is an integer} \\ X_{(\lfloor pR \rfloor + 1)} & \text{otherwise} \end{cases}$$

For simplicity, we use $\hat{q}_p = X_{(\lfloor pR \rfloor)}$.

4.3 Estimators of a Quantile II

Proposition 4.3.1 – Asymptotic Normality of $\hat{q}_p$

Assume f is continuous and positive in a neighborhood of q_p . Then:

- (i) $\hat{q}_p$ is a strongly consistent estimator of q_p
- (ii) $\sqrt{R}(\hat{q}_p - q_p) \xrightarrow{\mathcal{D}} \mathcal{N}(0, \varsigma^2)$, where $\varsigma^2 = \frac{p(1-p)}{f(q_p)^2}$

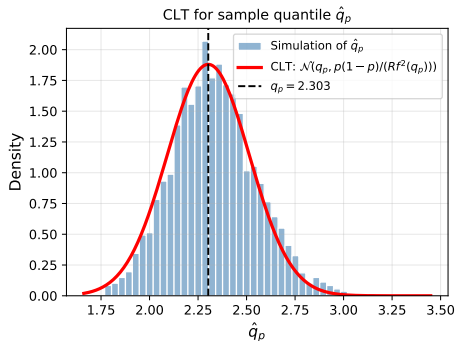
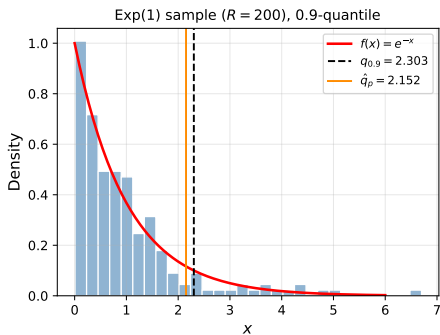
Symbols: $f(q_p)$ = density at the quantile, p = probability level.

4.3 Estimators of a Quantile III

Proof sketch: Based on the Beta distribution of order statistics.

- $U_{(\lfloor Rp \rfloor)} \sim \text{Beta}(\lfloor Rp \rfloor, R - \lfloor Rp \rfloor)$
- By Lemma 4.3.3: $\sqrt{R}(U_{\lfloor Rp \rfloor} - p) \xrightarrow{\mathcal{D}} \mathcal{N}(0, p(1 - p))$
- Apply Taylor expansion: $F^{\leftarrow}(p + x) \approx F^{\leftarrow}(p) + \frac{x}{f(q_p)}$
- Combining: $\sqrt{R}(\hat{q}_p - q_p) \xrightarrow{\mathcal{D}} \mathcal{N}(0, p(1 - p)/f(q_p)^2)$

4.3 Estimators of a Quantile IV



4.3.1 Estimation of $1/f(q_p)$ via Order Statistics I

To construct a practical CI for q_p , we need to estimate $1/f(q_p)$.

Kernel Estimator (Proposition 4.3.4)

Define $Q(p) = F^{-1}(p)$, so $Q'(p) = 1/f(Q(p)) = 1/f(q_p)$.

Using finite differences with parameter $m = m_R$:

$$\hat{Q}'_{p,R} = \frac{R}{2m} (X_{(\lfloor pR \rfloor + m)} - X_{(\lfloor pR \rfloor - m + 1)})$$

Consistency: If f has a continuous derivative near q_p and $m_R \rightarrow \infty$ with $m_R = o(R)$, then $\hat{Q}'_{p,R}$ is strongly consistent.

Choice of bandwidth m_R :

- Simple rule: $m_R = cR^{1/2}$
- MSE-optimal (Bloch-Gastwirth): $m_R = cR^{4/5}$ minimizes $\mathbb{E}(\hat{Q}'_{p,R} - Q'(p))^2 \sim \frac{5}{4} \cdot 9^{-1/5} [Q'(p)]^{4/5} [Q''(p)]^{3/5} R^{-2/5}$

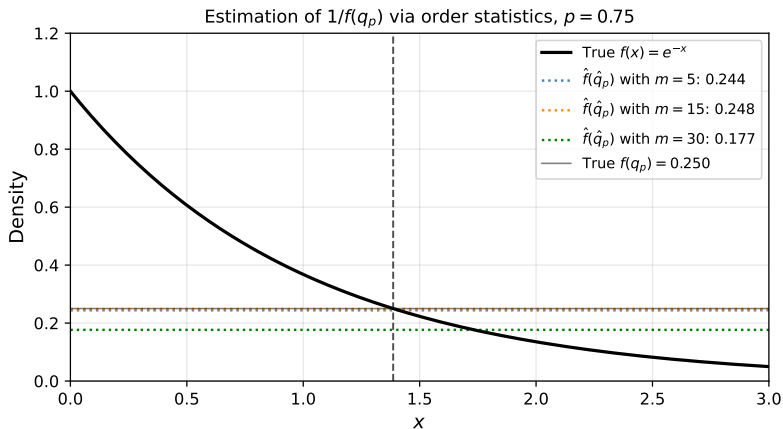
4.3.1 Estimation of $1/f(q_p)$ via Order Statistics II

Confidence Interval for q_p

Combining Proposition 4.3.1 and $\hat{Q}'_{p,R}$:

$$\hat{q}_p \pm z_{1-\alpha/2} \sqrt{\frac{p(1-p)}{R}} \hat{Q}'_{p,R}$$

4.3.1 Estimation of $1/f(q_p)$ via Order Statistics III



Example: $X \sim \text{Exp}(1)$, $p = 0.75$, $R = 500$.

4.3.1 Estimation of $1/f(q_p)$ via Order Statistics IV

- True quantile: $q_{0.75} = \ln 4 \approx 1.386$
- True $f(q_p) = e^{-q_{0.75}} = 0.25$
- Different bandwidth choices m give different accuracy
- Larger m : more smoothing, potentially more bias

4.3 Python: Sample Quantile and Confidence Interval

```
import numpy as np
from scipy import stats

def quantile_ci(X_samples, p, m=None, alpha=0.05):
    """
    Estimate p-quantile and compute confidence interval.

    Parameters
    -----
    X_samples : 1D array of i.i.d. samples
    p          : quantile level in (0, 1)
    m          : bandwidth parameter (default: sqrt(R))
    """
    R = len(X_samples)
    X_sorted = np.sort(X_samples)
    idx = int(np.floor(p * R))          # index of sample quantile
    q_hat = X_sorted[idx]              # sample quantile

    if m is None:
        m = max(1, int(np.sqrt(R)))

    lo_idx = max(0, idx - m)
    hi_idx = min(R - 1, idx + m - 1)
    # Estimate 1/f(q_p) via order-stat differences
    Q_prime_hat = R * (X_sorted[hi_idx] - X_sorted[lo_idx]) / (2 * m)

    # Confidence interval
```

4.4 More on Absolute vs Relative Errors I

Recall the error conditions:

$$\text{Absolute: } \mathbb{P}(|\hat{I} - I| \geq \varepsilon) \leq \delta$$

$$\text{Relative: } \mathbb{P}(|\hat{I}/I - 1| \geq \varepsilon) \leq \delta$$

Key Insight: Dependence on I

From the CLT-based fundamental relationship:

- For **absolute error**: $R = \frac{z_{1-\delta/2}^2 \text{Var}(Y)}{\varepsilon^2}$ (independent of I !)
- For **relative error**: $R = \frac{z_{1-\delta/2}^2 \text{Var}(Y)}{\varepsilon^2 I^2}$ (grows as $I \rightarrow 0$)

Achieving small relative error becomes *increasingly expensive* when I is small.

This motivates the use of variance reduction techniques (Chapter 5) to control both error types.

4.4.1 Chernoff Bound I

Proposition 4.4.1 – Chernoff Bound

Let $S_R = Y_1 + \dots + Y_R$ where Y_i i.i.d. Bernoulli(p). For any $0 < \varepsilon < 1$:

$$\text{Upper tail: } \mathbb{P}(S_R \geq (1 + \varepsilon)Rp) \leq \exp\left(-\frac{\varepsilon^2 Rp}{2 + \varepsilon}\right) \leq \exp\left(-\frac{\varepsilon^2 Rp}{3}\right)$$

$$\text{Lower tail: } \mathbb{P}(S_R \leq (1 - \varepsilon)Rp) \leq \exp\left(-\frac{\varepsilon^2 Rp}{2}\right)$$

Combining:

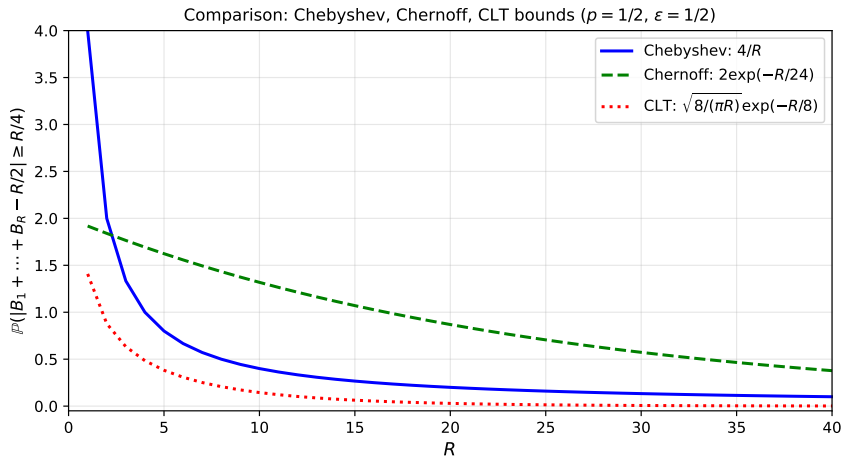
$$\mathbb{P}(|S_R - Rp| \geq \varepsilon Rp) \leq 2 \exp\left(-\frac{\varepsilon^2 Rp}{3}\right)$$

4.4.1 Chernoff Bound II

Corollary 4.4.2 – Required Replications for Relative Error

For $\hat{Y}_R = \frac{1}{R} \sum_{i=1}^R Y_i$ with $Y_i \sim \text{Bernoulli}(p)$: If $R \geq \frac{3 \ln(2/\delta)}{\varepsilon^2 p}$, then $\mathbb{P}(|\hat{Y}_R - p| \geq \varepsilon p) \leq \delta$.

4.4.1 Chernoff Bound III



4.4.1 Chernoff Bound IV

Comparison for $\mathbb{P}(|B_1 + \dots + B_R - R/2| \geq R/4)$, $p = 1/2$, $\varepsilon = 1/2$:

- **Chebyshev**: $4/R$ (loosest, decreases slowly)
- **Chernoff**: $2e^{-R/24}$ (exponential decay)
- **CLT**: $\sqrt{8/(\pi R)} e^{-R/8}$ (tightest, best approximation)

4.4.2 Discussion on Absolute vs Relative Errors I

Counter-intuitive scenario: Suppose $\mathbb{P}(Y = 1) = p = 0.01$, $R = 1$. Then $\hat{Y}_1 = Y_1 \in \{0, 1\}$.

- $\mathbb{P}(|\hat{Y}_1 - I| \leq 0.01) = \mathbb{P}(Y_1 = 0) = 0.99$
- This “satisfies” absolute error condition (E1) with $\varepsilon = \delta = 0.01$!
- But relative error is huge: $|\hat{Y}_1 - I|$ is either 0.01 or 0.99

Required replications ($\varepsilon = \delta = 0.01$) for absolute error:

I	Required R
0.1	≈ 5945
0.01	≈ 654
0.001	≈ 66

Smaller $p \Rightarrow$ fewer replications for absolute error!

4.4.2 Discussion on Absolute vs Relative Errors II

Required replications for relative error (Chernoff, $\varepsilon = \delta = 0.01$):

I	Required R
0.1	$\approx 1.59 \times 10^6$
0.01	$\approx 1.59 \times 10^7$
0.001	$\approx 1.59 \times 10^8$

$R \propto 1/I$ for relative error – **extremely expensive** when I is small!

Implication

When I is small (e.g., rare event probability), achieving small relative error via naive Monte Carlo requires an impractically large number of replications. Variance reduction techniques (importance sampling, etc.) are essential.

4.4.3 Absolute vs Relative Error for Estimating π I

Setup: Estimator $\hat{Y}_{R^{\text{abs}}}$ from Example 4.1.6: $Y_j = 4 \cdot \mathbf{1}(U_{2j-1}^2 + U_{2j}^2 \leq 1)$, $\mathbb{E}Y = \pi$, $\text{Var } Y \approx 2.6968$.

Absolute Error (CLT)

To achieve $\mathbb{P}(|\hat{Y}_{R^{\text{abs}}} - \pi| \leq 0.01) \geq 0.99$ ($\varepsilon = \delta = 0.01$, $z_{1-\delta/2} = 2.57$):

$$R^{\text{abs}} = z_{1-\delta/2}^2 \frac{\text{Var } Y}{\varepsilon^2} \approx 2.57^2 \cdot \frac{2.6968}{0.01^2} \approx 1.78 \times 10^5$$

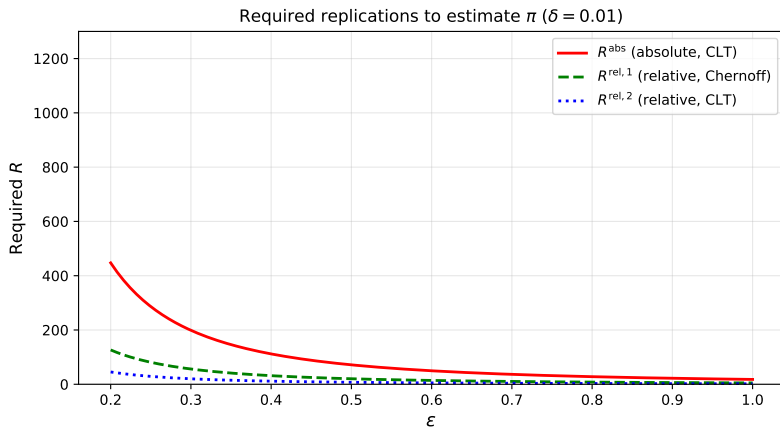
Relative Error (Chernoff)

To achieve $\mathbb{P}(|\hat{Y}_{R^{\text{rel},1}} - \pi| \leq 0.01\pi) \geq 0.99$:

$$R^{\text{rel},1} \geq \frac{3 \ln(2/0.01)}{0.01^2 \cdot \pi} \approx 5 \times 10^4$$

Via CLT (setting $\varepsilon' = 0.01/\pi$): $R^{\text{rel},2} \approx 5 \times 10^5$

4.4.3 Absolute vs Relative Error for Estimating π II



For $\pi \approx 3.14$, relative error requires $\sim 3\times$ more replications.

4.4 Python: Comparing Error Bounds

```
import numpy as np
from scipy import stats

def required_R_absolute(sigma2, epsilon, delta):
    """R for absolute error via CLT."""
    z = stats.norm.ppf(1 - delta / 2)
    return z**2 * sigma2 / epsilon**2

def required_R_relative_chernoff(p, epsilon, delta):
    """R for relative error via Chernoff (Bernoulli only)."""
    return 3 * np.log(2 / delta) / (epsilon**2 * p)

def required_R_relative_clt(sigma2, I, epsilon, delta):
    """R for relative error via CLT."""
    z = stats.norm.ppf(1 - delta / 2)
    return z**2 * sigma2 / (epsilon**2 * I**2)

# Parameters for estimating pi
VarY_pi = 16 * (np.pi/4) * (1 - np.pi/4)    # indicator estimator
I_pi = np.pi
delta = 0.01
epsilon = 0.01

R_abs = required_R_absolute(VarY_pi, epsilon, delta)
R_rel1 = required_R_relative_chernoff(np.pi/4, epsilon, delta)
R_rel2 = required_R_relative_clt(VarY_pi, I_pi, epsilon, delta)
```

4.5 Conclusions I

Fundamental Relationship Revisited

The half-width of the confidence interval satisfies:

$$b = \frac{z_{1-\alpha/2} D}{\sqrt{R}}$$

where D depends on the problem:

- $I = \mathbb{E}Y$: $D = \sigma_Y$
- $z = k(I)$, $I = \mathbb{E}Y$: $D = |k'(I)| \sigma_Y$
- $z = k(\mathbf{I})$, $\mathbf{I} = \mathbb{E}\mathbf{Y}$: $D = \sqrt{\nabla k(\mathbf{I}) \Sigma_{\mathbf{Y}} (\nabla k(\mathbf{I}))^T}$

Practical workflow:

- 1 Run pilot simulation of size m to estimate σ_Y^2 (or $\Sigma_{\mathbf{Y}}$)
- 2 Compute required R from fundamental relationship

4.5 Conclusions II

- ③ Run R replications; compute $\hat{Y}_R$ (or $\hat{\mathbf{Y}}_R$) and $\hat{S}_R^2$
- ④ Report estimate with CI

4.5 Conclusions III

Summary of confidence intervals:

Quantity	Point Estimate	95% CI
I	$\hat{Y}_R$	$\hat{Y}_R \pm 1.96\hat{S}_R/\sqrt{R}$
$k(I)$	$k(\hat{Y}_R)$	$k(\hat{Y}_R) \pm 1.96\hat{S}_R k'(\hat{Y}_R) /\sqrt{R}$
q_p	$X_{(\lfloor pR \rfloor)}$	$\hat{q}_p \pm 1.96\sqrt{p(1-p)/R\hat{Q}'_{p,R}}$

Key takeaways:

- Error $b = O(R^{-1/2})$: the *Monte Carlo rate*
- CLT gives $\approx 80\%$ fewer replications than Chebyshev for same confidence
- Relative error control is much harder when I is small (rare events)
- Biased estimators: bias is $O(R^{-1})$, dominated by std. dev. $O(R^{-1/2})$ for large R
- For quantiles: $\varsigma^2 = p(1-p)/f(q_p)^2$ – need to estimate $f(q_p)$

4.5 Python: Full Output Analysis Workflow

```
import numpy as np
from scipy import stats

def full_mc_analysis(simulate_Y, R_pilot=500, R_main=None,
                    b=0.05, alpha=0.05, seed=42):
    """Complete MC output analysis pipeline."""
    rng = np.random.default_rng(seed)

    # Step 1: Pilot simulation to estimate sigma_Y^2
    Y_pilot = np.array([simulate_Y(rng) for _ in range(R_pilot)])
    sigma2_hat = Y_pilot.var(ddof=1)
    print(f"Pilot sigma^2 estimate: {sigma2_hat:.4f}")

    # Step 2: Required R from fundamental relationship
    z = stats.norm.ppf(1 - alpha / 2)
    if R_main is None:
        R_main = int(np.ceil(z**2 * sigma2_hat / b**2))
    print(f"Required R for b={b}, alpha={alpha}: {R_main}")

    # Step 3: Main simulation
    Y_main = np.array([simulate_Y(rng) for _ in range(R_main)])
    Y_bar = Y_main.mean()
    S = Y_main.std(ddof=1)

    ci_lo = Y_bar - z * S / np.sqrt(R_main)
    ci_hi = Y_bar + z * S / np.sqrt(R_main)
    print(f"Estimate: {Y_bar:.4f}, CI: ({ci_lo:.4f}, {ci_hi:.4f})")
```

Chapter 4: Summary

Estimation of $I = \mathbb{E}Y$:

- Unbiased estimator $\hat{Y}_R = \frac{1}{R} \sum Y_j$
- CLT: $\sqrt{R}(\hat{Y}_R - I) \rightarrow \mathcal{N}(0, \sigma_Y^2)$
- CI: $\hat{Y}_R \pm z_{1-\alpha/2} \hat{S}_R / \sqrt{R}$
- Required $R = z_{1-\alpha/2}^2 \sigma_Y^2 / b^2$
- 3 shapes of conf. regions: ellipsoid, parallelogram, rectangle

Estimation of $k(I)$:

- Delta method: $\zeta^2 = k'(I)^2 \sigma_Y^2$
- Generally biased; bias = $O(R^{-1})$
- $\text{MSE} = \text{Var} + \text{Bias}^2$

Quantile estimation:

- $\hat{q}_p = X_{(\lfloor pR \rfloor)}$: consistent
- CLT: $\zeta^2 = p(1-p)/f(q_p)^2$
- Estimate $1/f(q_p)$ via order statistics

Error bounds:

- Chebyshev: $b_R = \sigma_Y / (\alpha^{1/2} \sqrt{R})$
- CLT: $b = 1.96 \sigma_Y / \sqrt{R}$
- Chernoff: exponential bound for Bernoulli
- CLT uses $\sim 19\%$ as many replications as Chebyshev
- Relative error expensive when I is small

Monte Carlo rate: $b = O(R^{-1/2})$ regardless of dimension d